arm

From “Fast Enough” to “Fast by Design”

The Evolution of CPython Performance

Diego Russo
PyCon Italia, 2026-05-29

PyCon Italia Quattro (2010)

Who am I

Diego Russo

diego@python.org

CPython Core Developer

- CPython JIT
- Runtime performance
- Compiler & interpreter internals

Principal Software Engineer

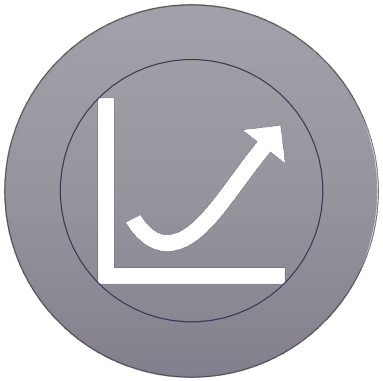
- Leading CPython work @ Arm
- Arm ecosystem enablement

Community & Conferences

- Europython organizer & speaker
- PyCon UK speaker

Where does Python spend time?

Why this talk



Python performance changed

The story is misunderstood

It is about trade-offs

0. Introduction

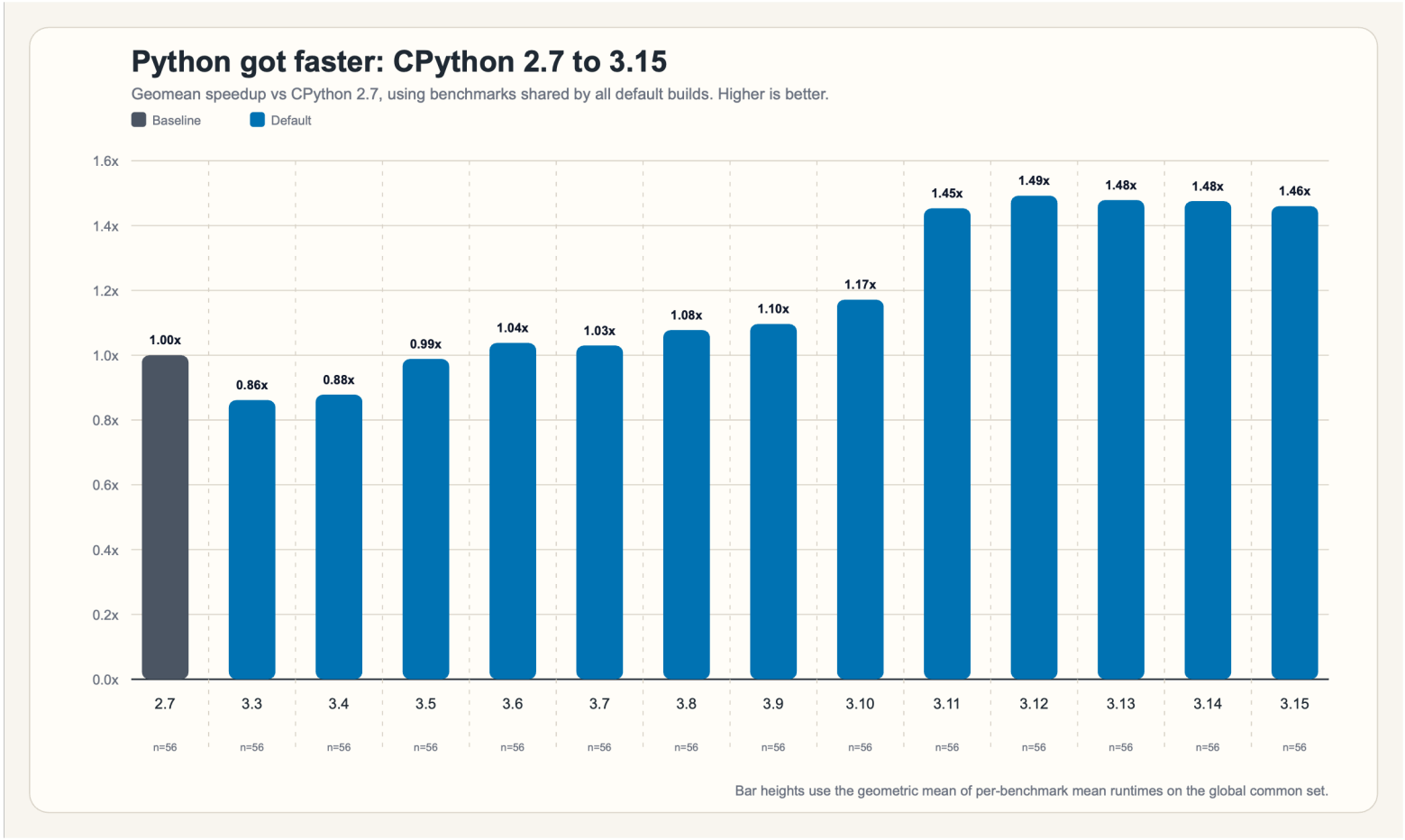
“Now is better than never”

-- The Zen of Python

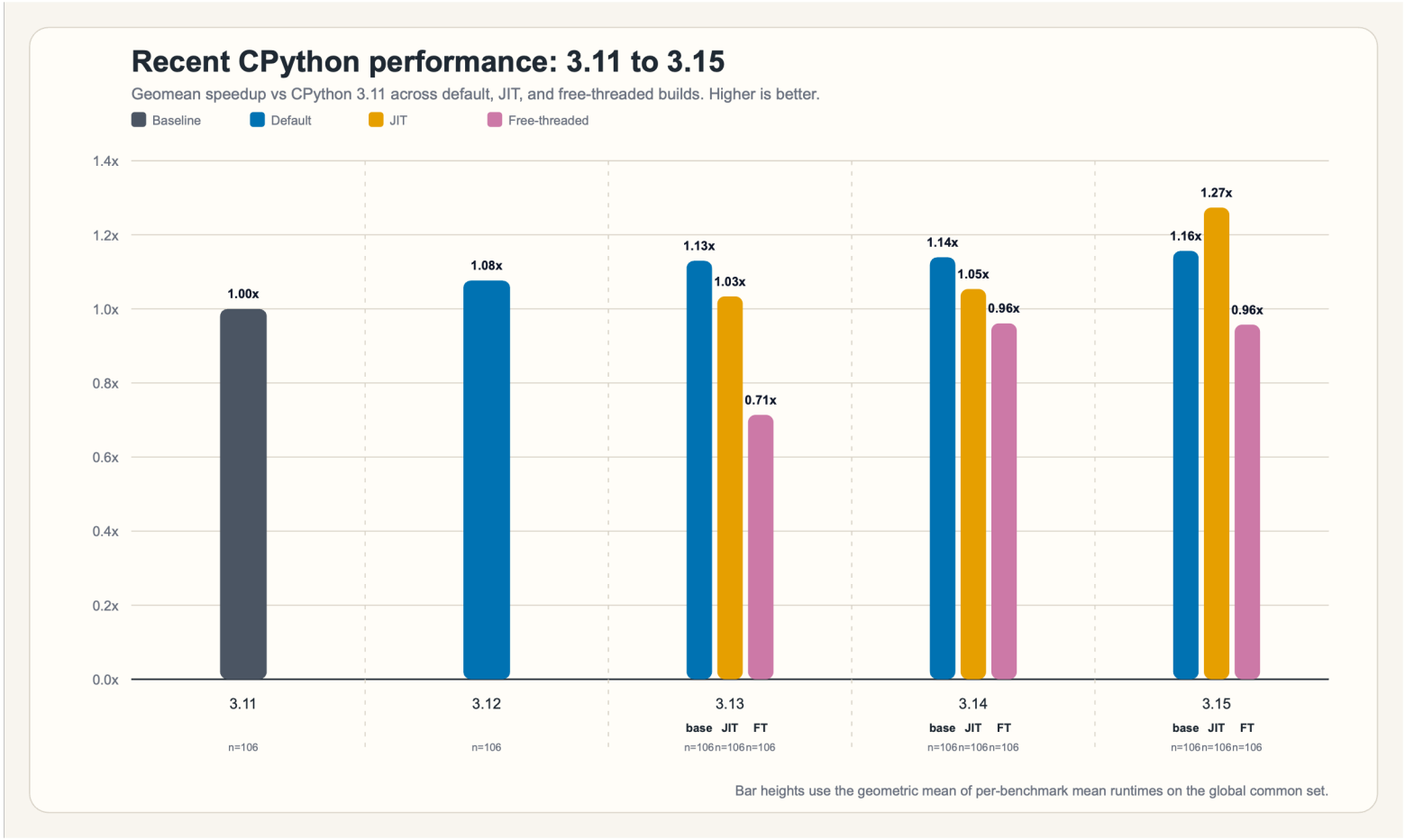
arm

A different performance landscape 

How fast is Python today?



The interpreter started adapting



This is still how many people think about Python

Python became optimizable by design 

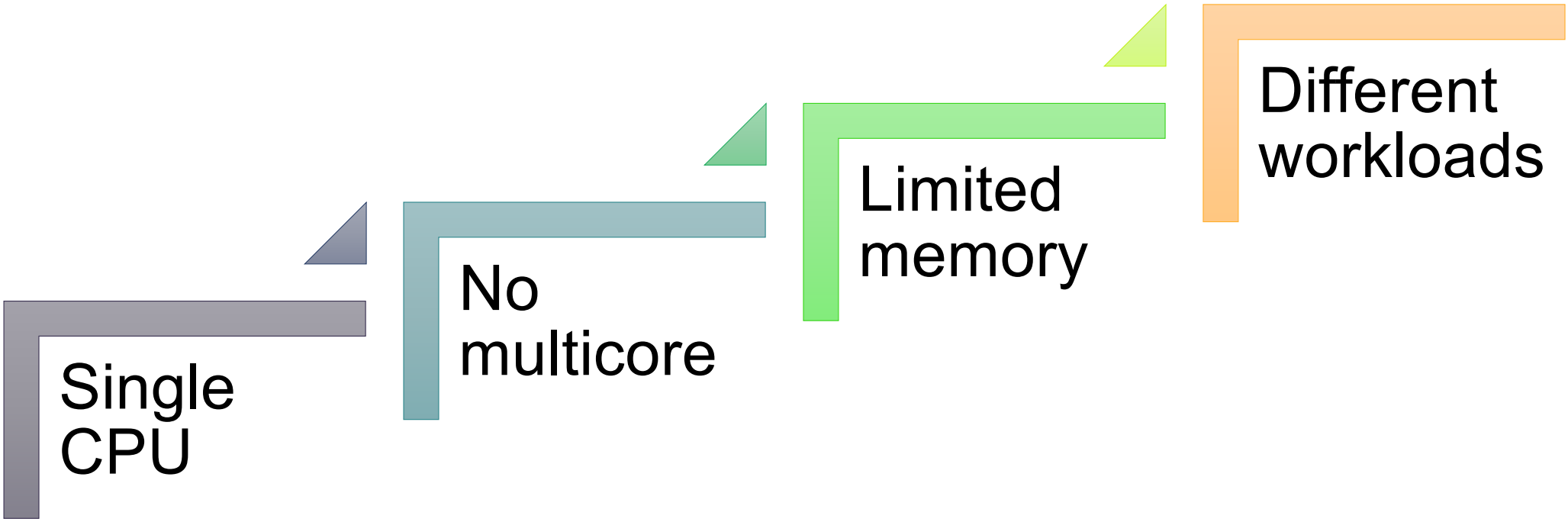
1. Original Design

“The programmer’s time is more important than the computer’s time.”

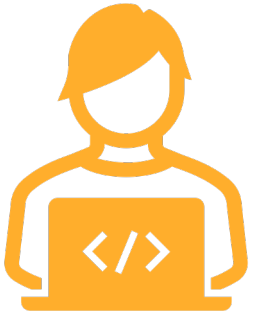
-- Guido van Rossum

What did performance mean in 1991? 

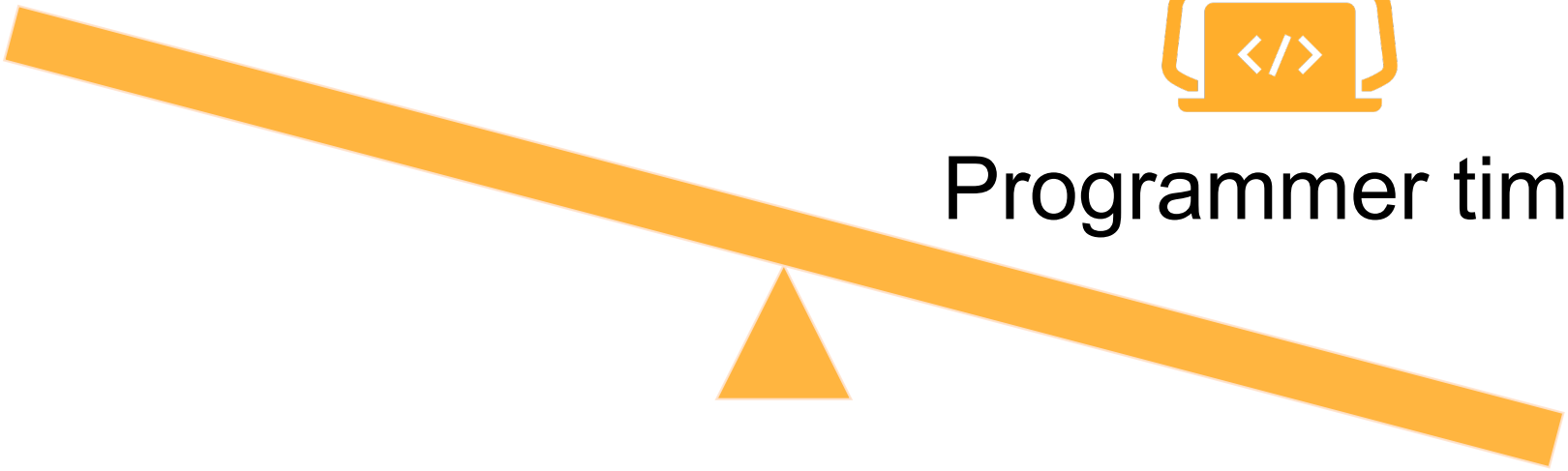
Different constraints, different priorities



Machine time

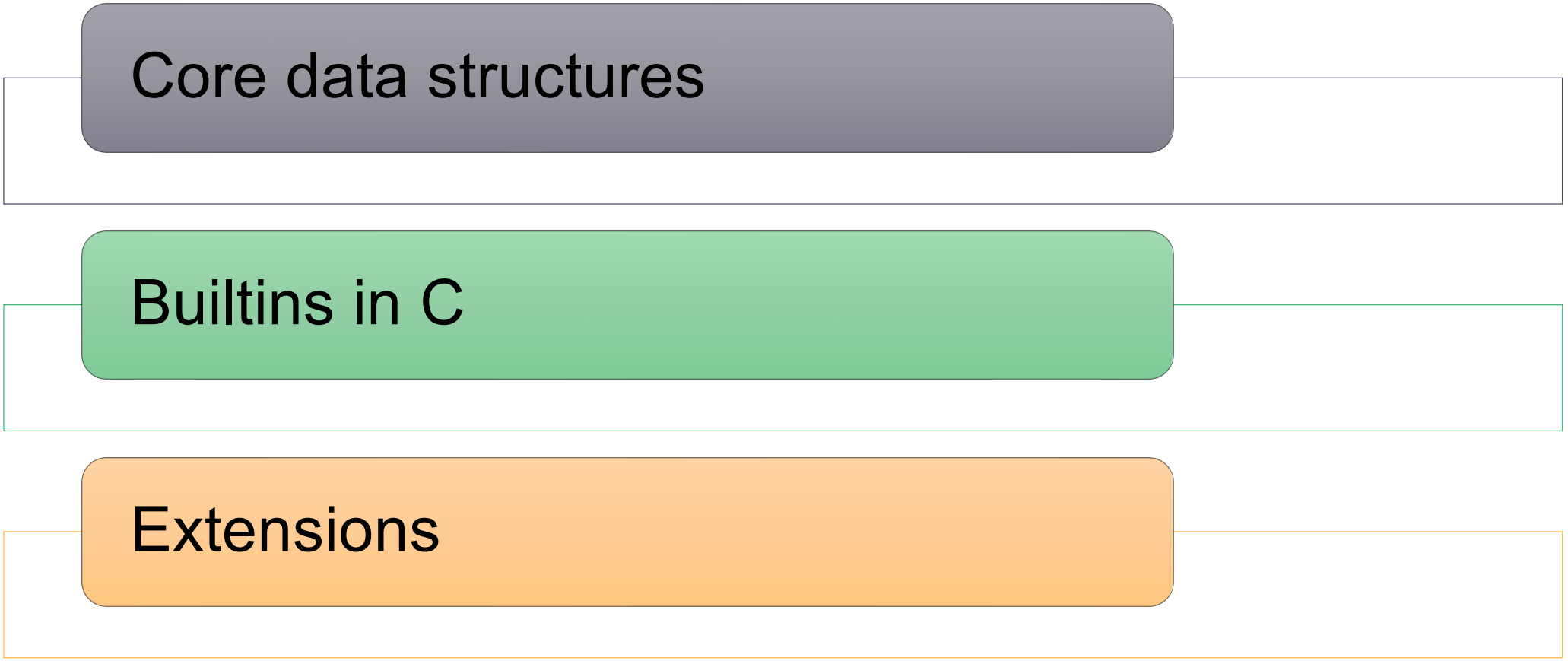


Programmer time



Everything is an object 

Performance lived in data structures and C



A deliberate trade-off ⚖️

2. The world changed

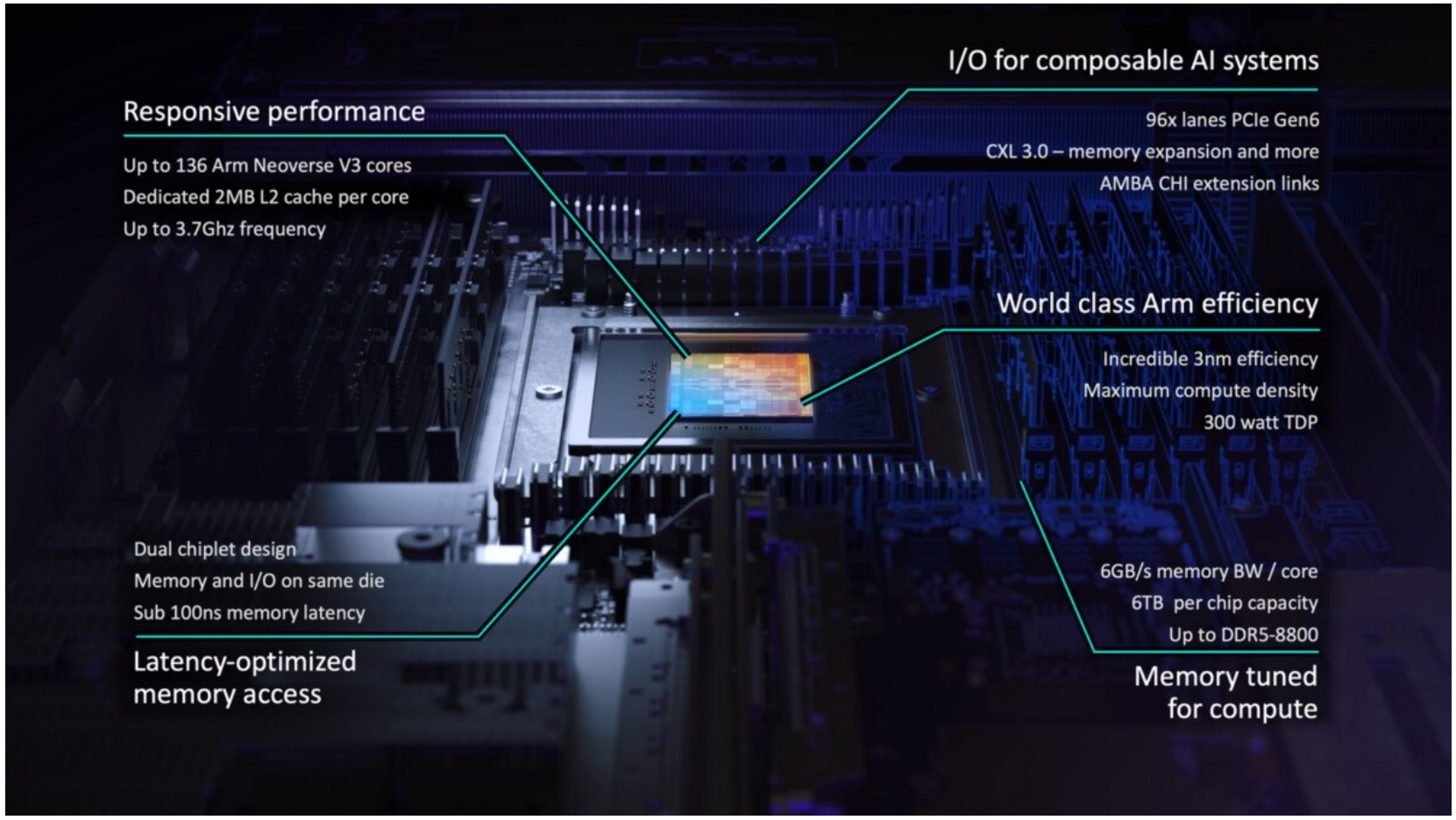
“Programs must be written for people to read, and only incidentally for machines to execute.”

-- Harold Abelson

Scripts

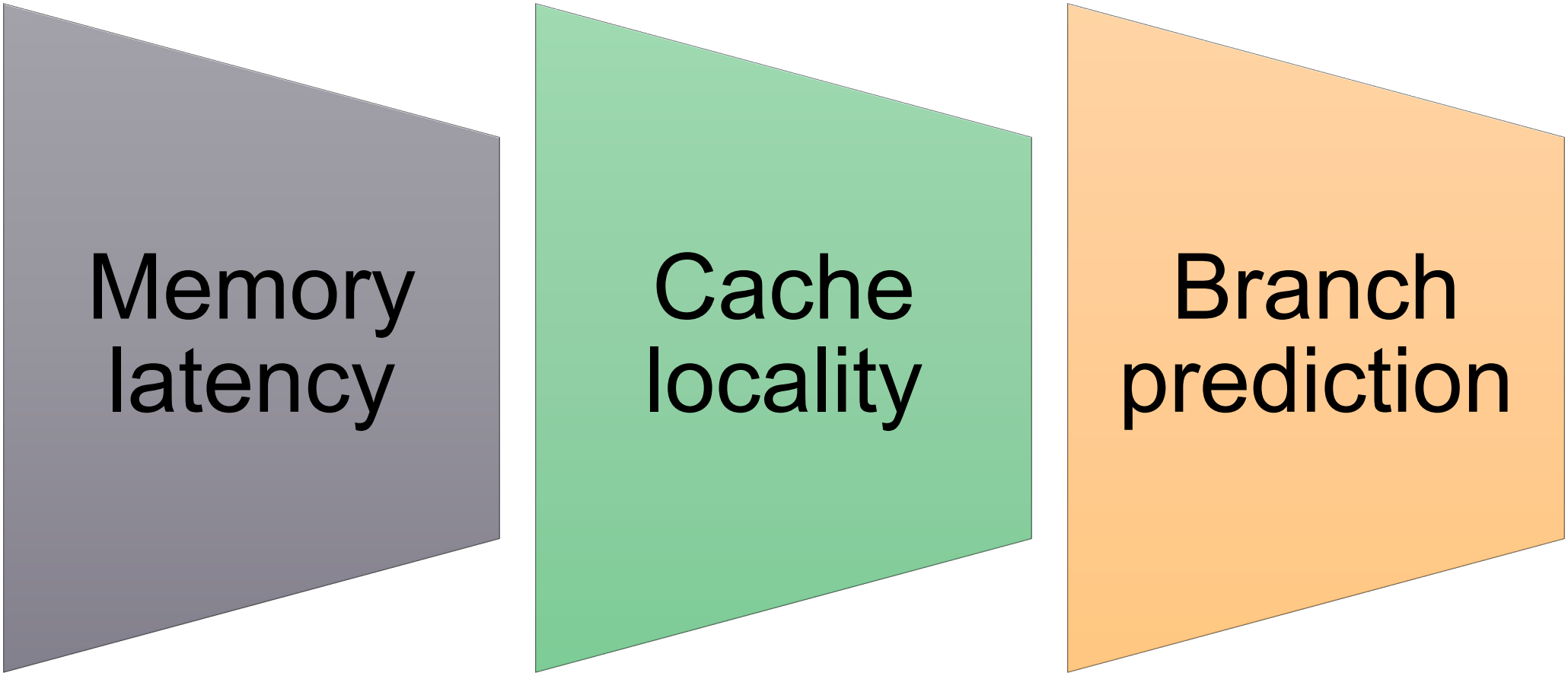
Infrastructure

Modern CPUs changed the rules

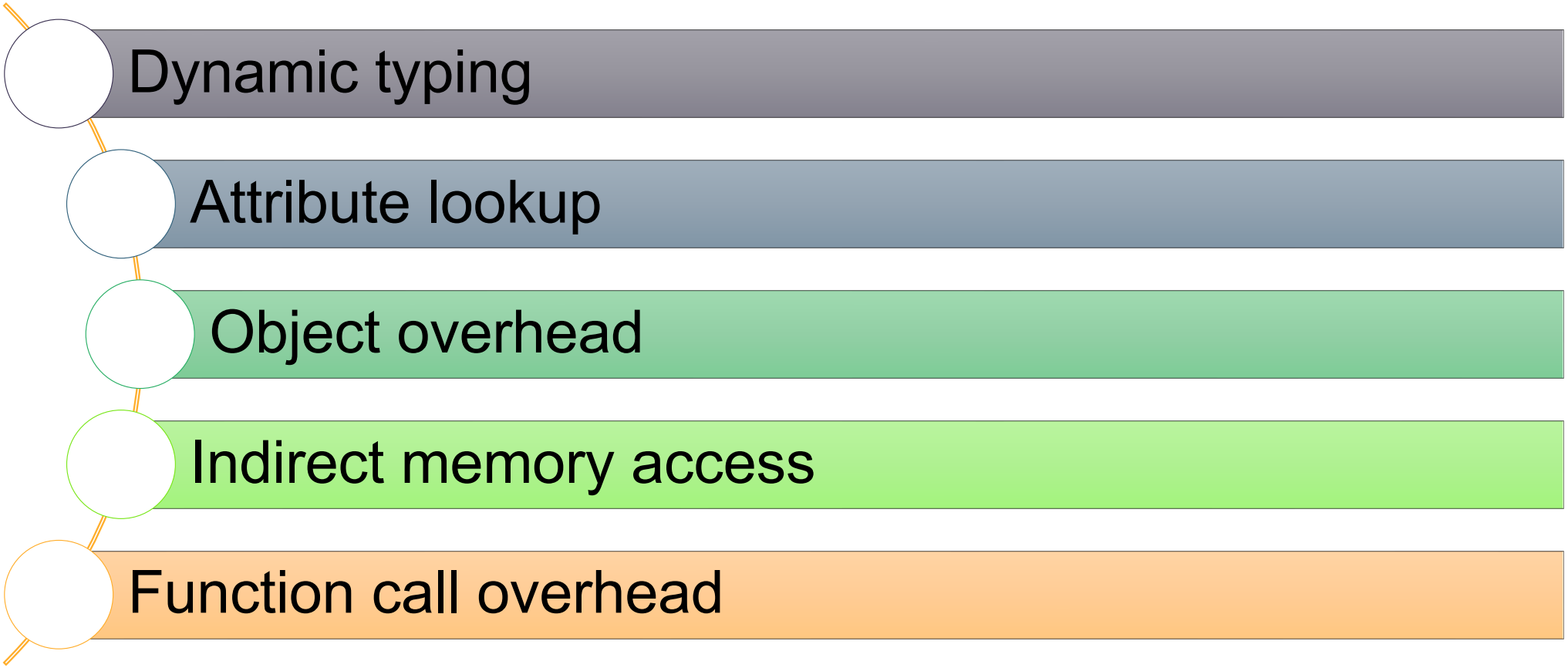


Modern CPUs changed the rules

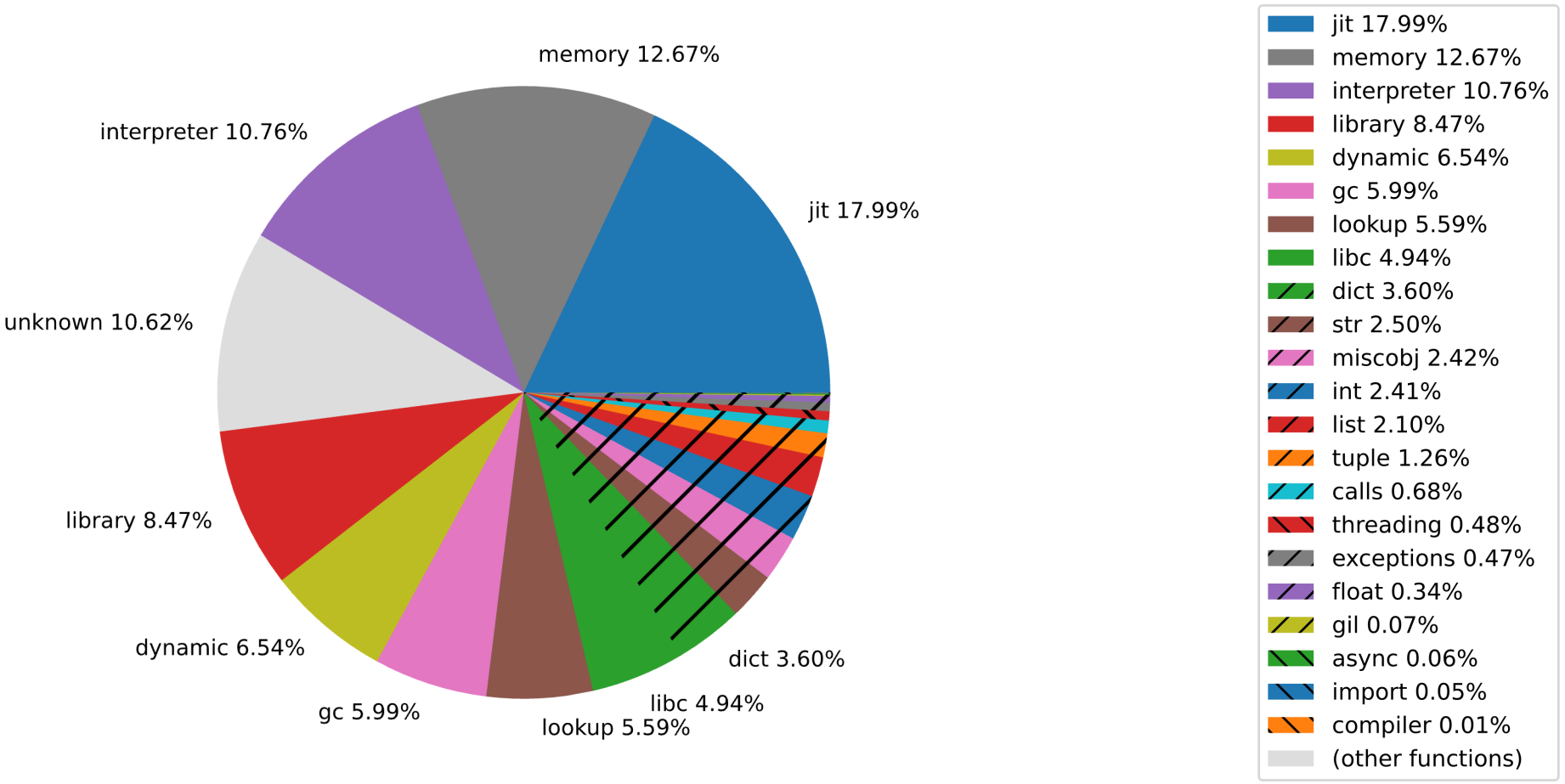
Memory and branches dominate performance



Why Python is slow (sometimes)



Performance is a system problem



3. Making CPython Optimizable

“Before you can optimize a system, you need to make it optimizable.”

-- Mark Shannon

arm

Before making it faster, make it optimizable 

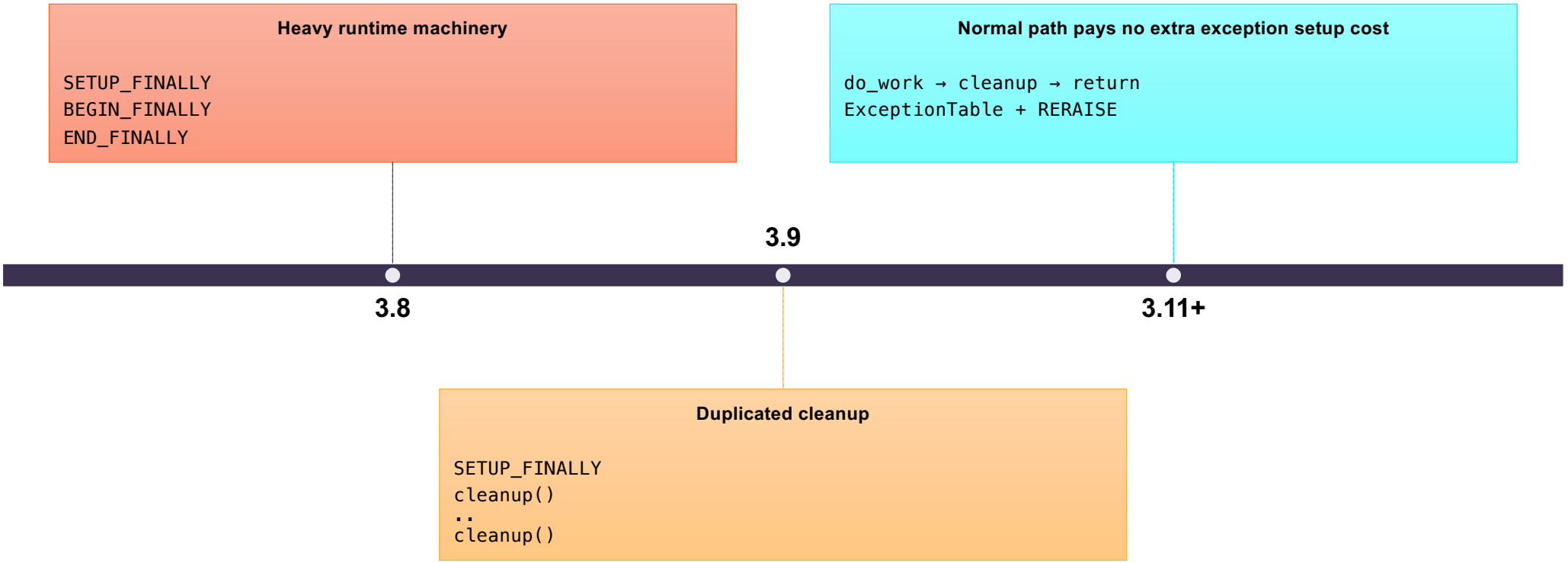
Too much work at runtime 

Move work earlier ⏪

try/finally: moving cost off the fast path

Same semantics. Less work on the common path.

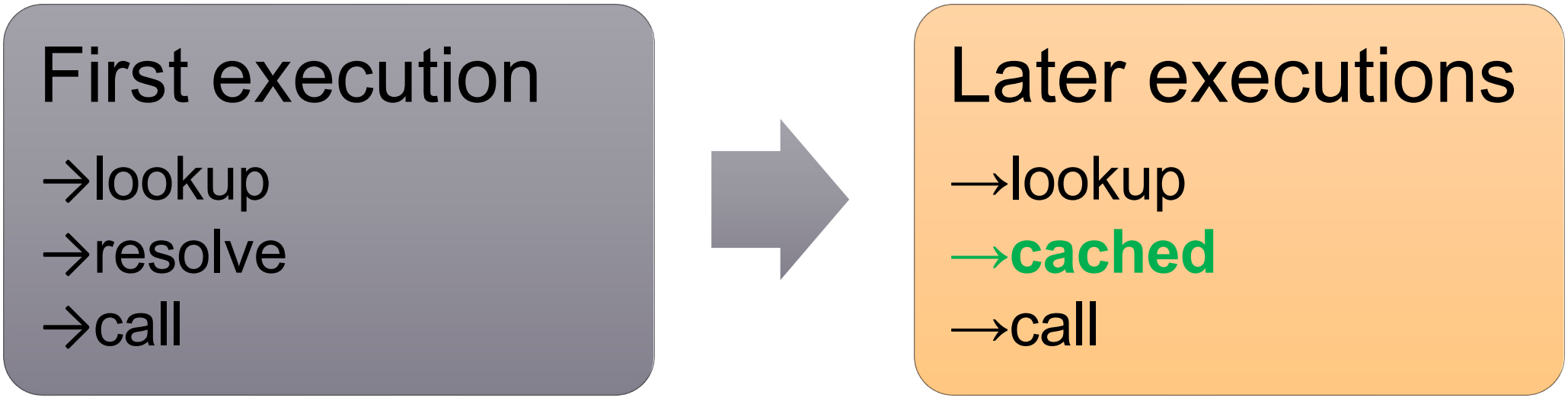
```
try:
    do_work()
finally:
    cleanup()
```



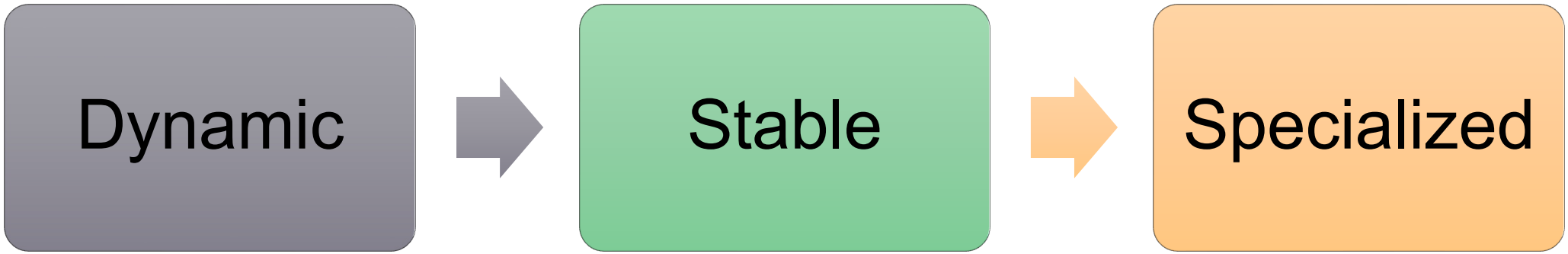
Stop asking the same question twice 😒

The specializing interpreter 

Inline caches and adaptive execution



From dynamic execution to specialized execution

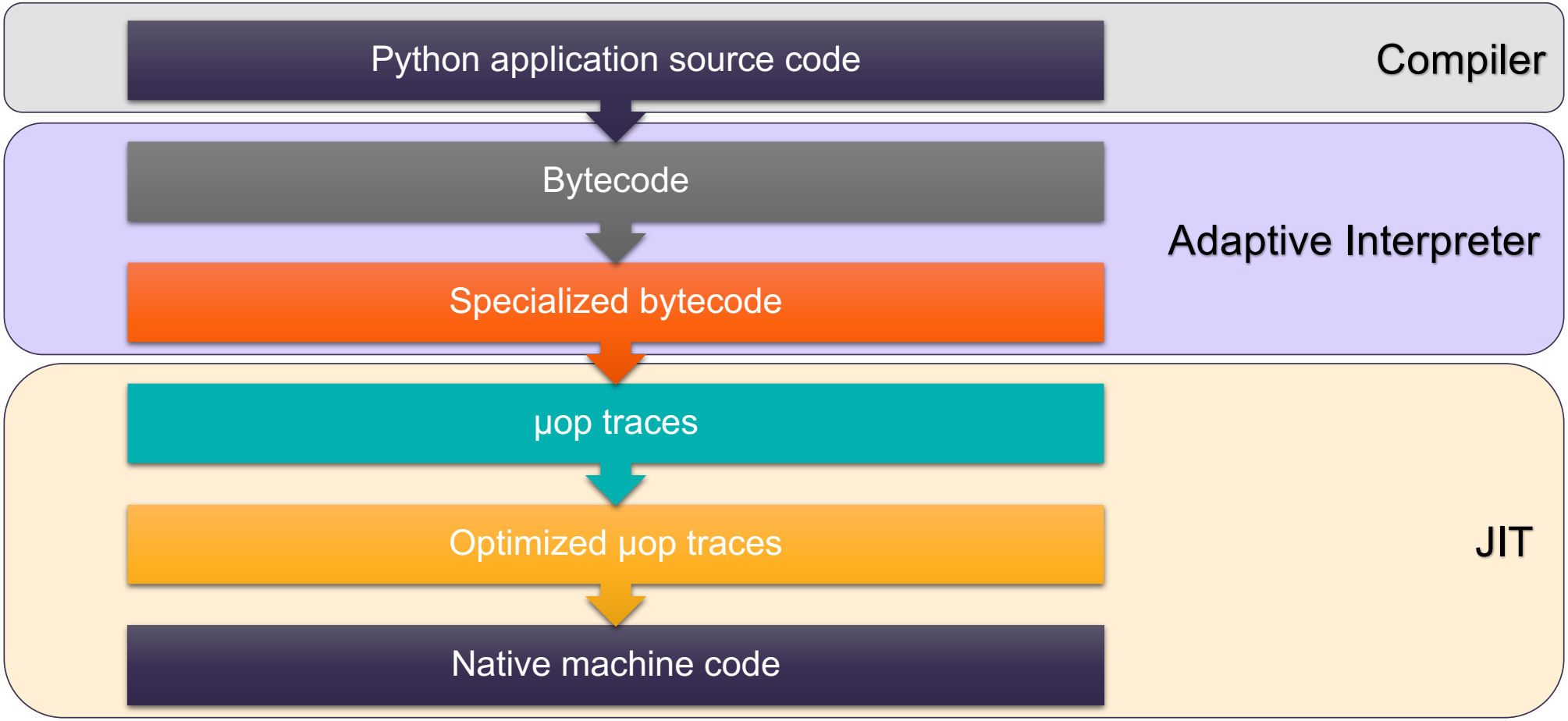


Interpretation and compilation align 🔗

The JIT compiler 🔥

From adaptive to compiled execution

How CPython executes and optimizes your code



Example: hot loop

Python source

```
def b():
    total = 0
    for i in range(10000):
        total += i
```

Python Bytecode

```
1           RESUME_CHECK             0
2           LOAD_SMALL_INT           0
            STORE_FAST               0 (total)
3           LOAD_GLOBAL              1 (range + NULL)
            LOAD_CONST               1 (10000)
            CALL                     1
            GET_ITER                 0
    L1:     FOR_ITER_RANGE          11 (to L2)
            STORE_FAST               1 (i)
4           LOAD_FAST_BORROW_LOAD_FAST_BORROW 1 (total, i)
            BINARY_OP_ADD_INT       13 (+=)
            STORE_FAST               0 (total)
            JUMP_BACKWARD_NO_JIT    13 (to L1)
3   L2:     END_FOR
            POP_ITER
            LOAD_CONST               2 (None)
            RETURN_VALUE
```

Example: hot loop

Python source

```
def b():
    total = 0
    for i in range(10000):
        total += i
```

Python Specialized Bytecode

```
1           RESUME_CHECK             0
2           LOAD_SMALL_INT           0
            STORE_FAST               0 (total)
3           LOAD_GLOBAL              1 (range + NULL)
            LOAD_CONST               1 (10000)
            CALL                     1
            GET_ITER                 0
    L1:     FOR_ITER_RANGE          11 (to L2)
            STORE_FAST               1 (i)
4           LOAD_FAST_BORROW_LOAD_FAST_BORROW 1 (total, i)
            BINARY_OP_ADD_INT       13 (+=)
            STORE_FAST               0 (total)
            JUMP_BACKWARD_NO_JIT    13 (to L1)
3   L2:     END_FOR
            POP_ITER
            LOAD_CONST               2 (None)
            RETURN_VALUE
```

Example: hot loop

µop trace

```
FOR_ITER_RANGE
    _CHECK_VALIDITY
    _SET_IP
    _ITER_CHECK_RANGE
    _GUARD_NOT_EXHAUSTED_RANGE
    _ITER_NEXT_RANGE
STORE_FAST
    _CHECK_VALIDITY
    _SET_IP
    _SWAP_FAST
    _POP_TOP
LOAD_FAST_BORROW_LOAD_FAST_BORROW
    _CHECK_VALIDITY
    _SET_IP
    _LOAD_FAST_BORROW
    _LOAD_FAST_BORROW
BINARY_OP_ADD_INT
    _CHECK_VALIDITY
    _SET_IP
    _GUARD_TOS_INT
    _GUARD_NOS_INT
    _BINARY_OP_ADD_INT
    _POP_TOP_INT
    _POP_TOP_INT
STORE_FAST
    _CHECK_VALIDITY
    _SET_IP
    _SWAP_FAST
    _POP_TOP
    _JUMP_TO_TOP
JUMP_BACKWARD
    _CHECK_VALIDITY
    _SET_IP
    _CHECK_PERIODIC
```

µop optimized trace

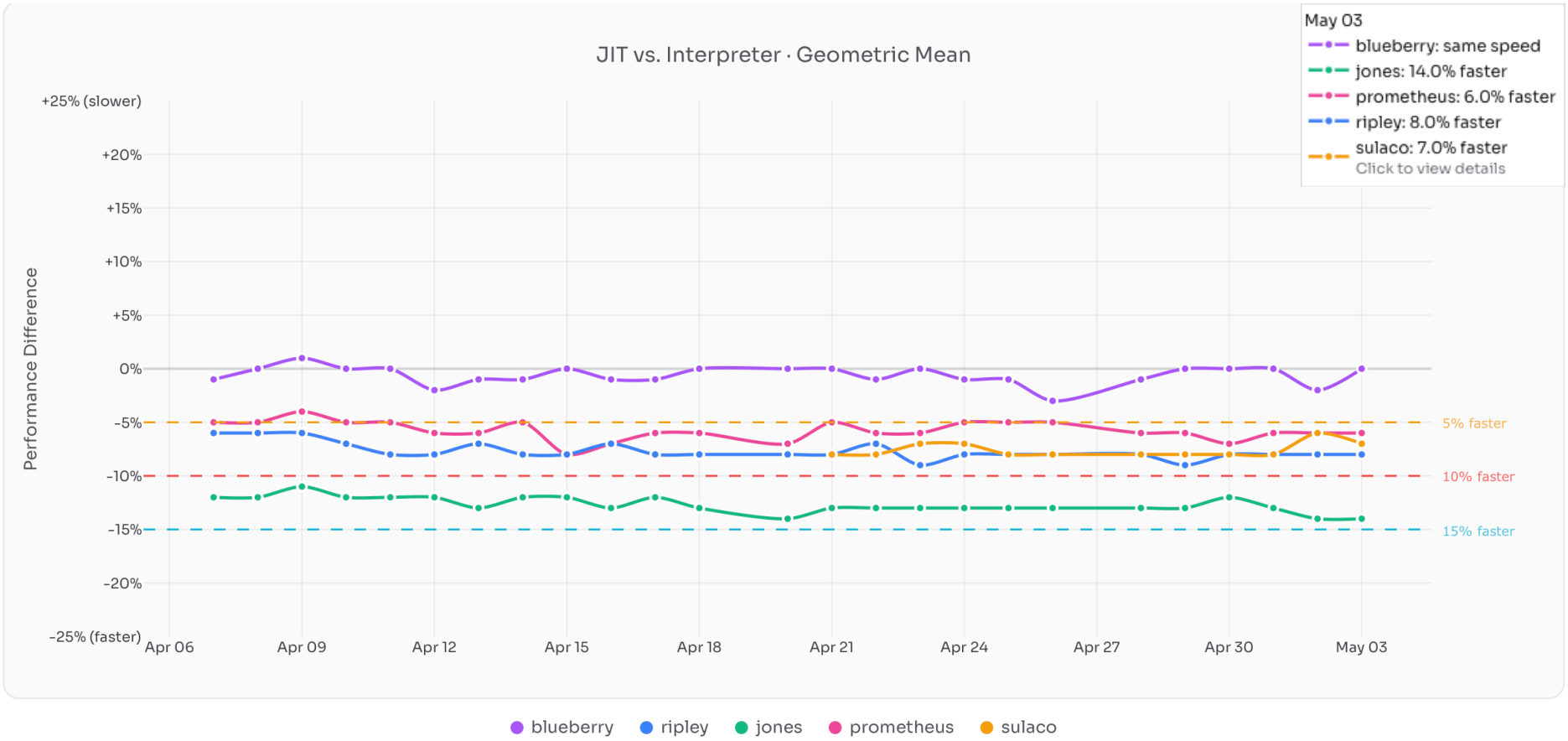
```
_CHECK_VALIDITY_r00
_ITER_CHECK_RANGE_r02
_GUARD_NOT_EXHAUSTED_RANGE_r22
_ITER_NEXT_RANGE_r23
_SET_IP_r33
_SWAP_FAST_1_r33
_SPILL_OR_RELOAD_r31
_POP_TOP_r10
_CHECK_VALIDITY_r00
_LOAD_FAST_BORROW_0_r01
_LOAD_FAST_BORROW_1_r12
_GUARD_TOS_OVERFLOWED_r22
_GUARD_NOS_INT_r22
_BINARY_OP_ADD_INT_r23
_POP_TOP_NOP_r32
_POP_TOP_NOP_r21
_SWAP_FAST_0_r11
_POP_TOP_INT_r10
_JUMP_TO_TOP_r00
```

Machine code

```
a8 82 5f f8 e9 03 28 2a 3f 05 40 f2 20 01 00 54
08 f9 7f 92 09 00 00 d0 08 05 40 f9 29 ed 46 f9
1f 01 09 eb 40 00 00 54 66 06 00 14 0d 00 00 14
fd 7b bf a9 08 00 00 d0 00 00 00 d0 00 80 35 91
08 f1 46 f9 01 00 00 d0 21 f4 35 91 03 00 00 d0
63 80 36 91 22 3f 80 52 fd 03 00 91 00 01 3f d6
fd 7b bf a9 a8 82 5f f8 fd 03 00 91 e9 03 28 2a
3f 05 40 f2 c0 01 00 54 08 f9 7f 92 0a 00 00 d0
09 05 40 f9 4a 75 47 f9 3f 01 0a eb 41 02 00 54
08 11 40 f9 1f 01 00 f1 6d 00 00 54 fd 7b c1 a8
18 00 00 14 fd 7b c1 a8 0c 07 00 14 08 00 00 d0
00 00 00 d0 00 d4 39 91 08 79 47 f9 01 00 00 d0
21 48 3a 91 03 00 00 d0 63 d4 3a 91 22 3f 80 52
00 01 3f d6 08 00 00 d0 00 00 00 d0 00 a0 37 91
08 79 47 f9 01 00 00 d0 21 20 38 91 03 00 00 d0
63 94 38 91 42 0f 80 52 00 01 3f d6 1f 20 03 d5
fd 7b bf a9 a8 82 5f f8 fd 03 00 91 e9 03 28 2a
3f 05 40 f2 e0 06 00 54 08 f9 7f 92 0a 00 00 f0
09 05 40 f9 4a 79 40 f9 3f 01 0a eb 61 07 00 54
09 11 40 f9 3f 01 00 f1 4d 08 00 91 62 0f 80 52
00 01 3f d6 08 00 00 f0 00 00 00 d0 00 ec 3d 91
08 85 40 f9 01 00 00 d0 21 60 3c 91 03 00 00 d0
63 ac 3c 91 e2 0f 80 52 00 01 3f d6 08 00 00 f0
00 00 00 d0 00 18 3e 91 08 85 40 f9 01 00 00 d0
21 60 3c 91 03 00 00 d0 63 ac 3c 91 42 11 80 52
00 01 3f d6 08 00 00 f0 00 00 00 f0 00 dc 00 91
08 85 40 f9 01 00 00 d0 21 e4 3e 91 03 00 00 f0
63 3c 00 91 62 3b 80 52 00 01 3f d6 08 00 00 f0
00 00 00 f0 00 48 01 91 08 85 40 f9 01 00 00 f0
21 d8 01 91 03 00 00 f0 63 70 02 91 a2 02 80 52
00 01 3f d6 08 00 00 f0 00 00 00 f0 00 48 03 91
```


What does the JIT actually give you?

https://www.doesjitgobrrr.com/



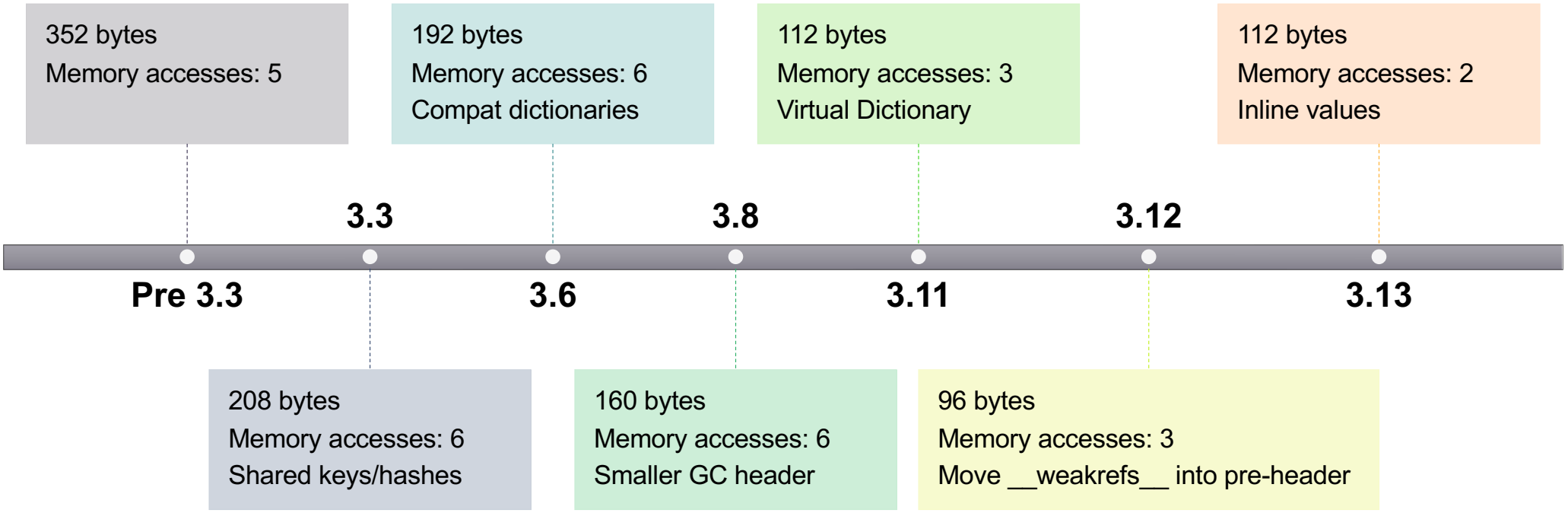
Why JIT is not a silver bullet 

Why this matches modern CPUs 

Object layout evolution

More memory → fewer indirections → fewer memory accesses

```
class C:
  def __init__(self, a, b):
    self.a = a
    self.b = b

c = C()
c.a
```



arm

Small changes, big effects

4. The GIL

“Simple is better than complex.”

-- Zen of Python

The Global Interpreter Lock 

The GIL

Early model, simplified

GIL implementation

```
static PyThread_type_lock interpreter_lock = 0;

void PyEval_InitThreads(){
    interpreter_lock = PyThread_allocate_lock();
    PyThread_acquire_lock(interpreter_lock, 1);
}

void PyEval_AcquireLock(){
    PyThread_acquire_lock(interpreter_lock, 1);
}

void PyEval_ReleaseLock(){
    PyThread_release_lock(interpreter_lock);
}
```

GIL use

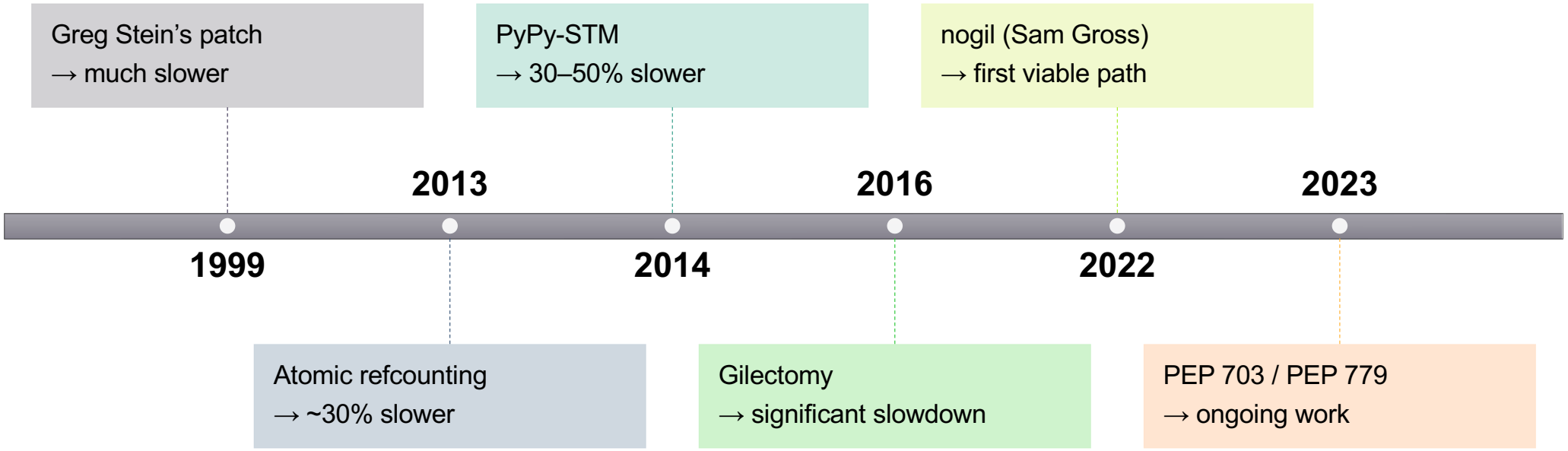
```
/* One thread at a time executes Python bytecode */
PyEval_AcquireLock();

while (running) {
    execute_next_bytecode();

    /* Periodically give other threads a chance */
    if (--ticker <= 0) {
        PyEval_ReleaseLock();
        /* other threads may run */
        PyEval_AcquireLock();
        ticker = check_interval;
    }
}
PyEval_ReleaseLock();
```


Why it made sense 💡

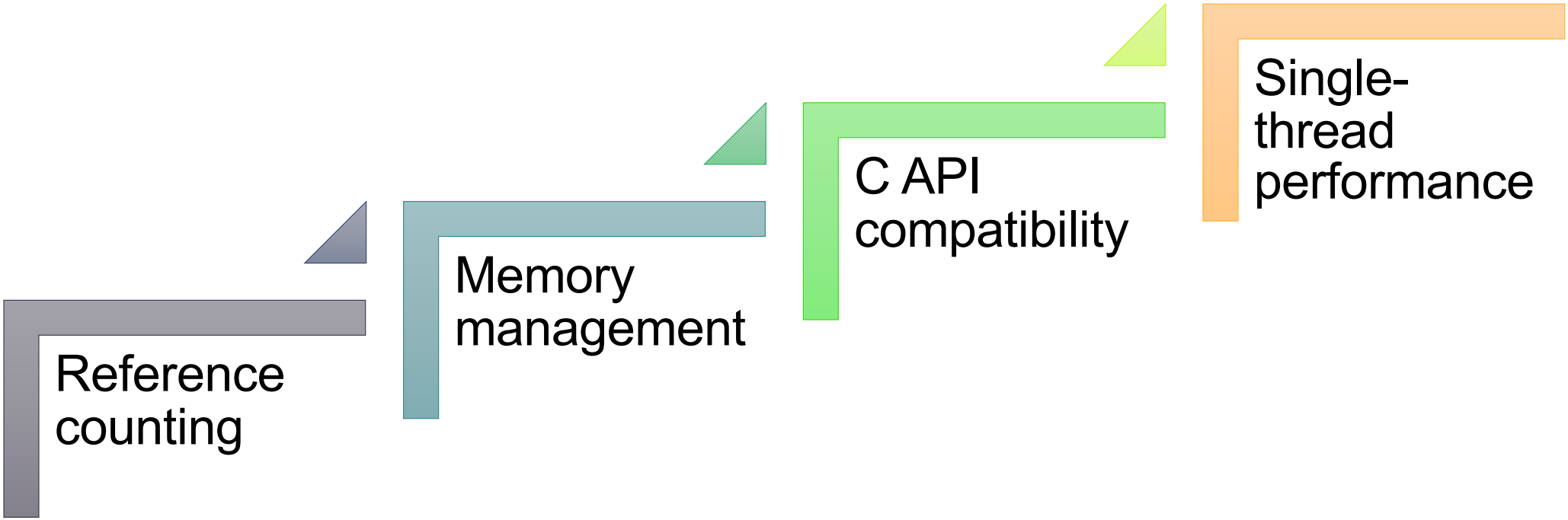
Why it became a problem 😬

This was tried before 

And failed (many times) 😓



Why it is hard



What changed recently 

Free-threading is not free 

5. Beyond the interpreter

“The fastest code is the code that doesn’t run.”

-- Robert Galanakis

Performance is more than execution speed ⚡

Startup time and lazy imports 🥱

Profiling and observability 

Async: a different kind of scalability 

6. The Cost

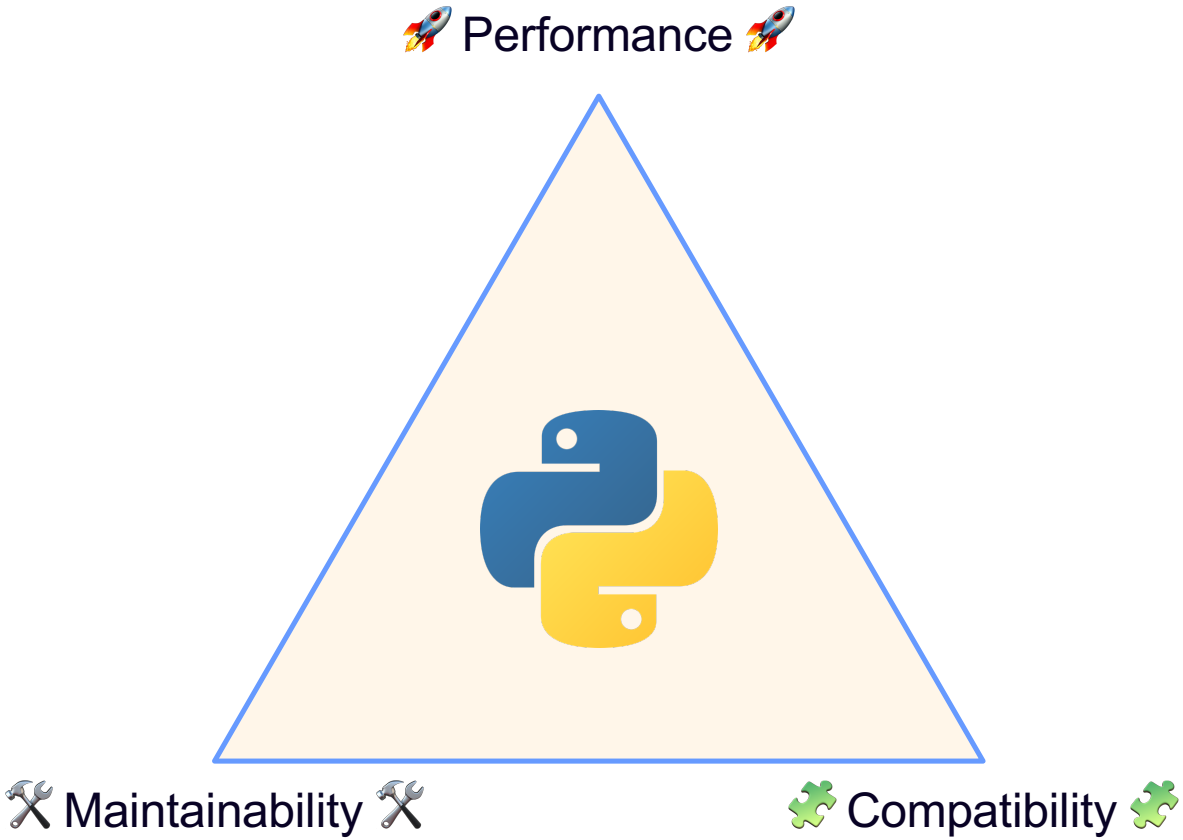
“Controlling complexity is the essence of computer programming.”

-- Brian Kernighan

Performance has a cost 

The code got more complex 

Trade-offs are real ⚖️



Performance requires investment 

7. The evolution

“Nothing in Python is very revolutionary.”

-- Guido van Rossum

Came for the language. Stayed for the community.

…and many CPython core developers and contributors 💙

Why this evolution matters 🌱

What did not change 😌

Same questions. Different scale.

PyCon Italia Quattro

PyCon Italia 2026

From “Fast Enough” to “Fast by Design” 🚀

arm

Tack
ಧನ್ಯವಾದಗಳು
Merci
Danke
Gracias
Grazie
谢谢
ありがとう
Asante
Thank you
감사합니다
धन्यवाद
Kiitos
شكرًا
ধন্যবাদ
תודה
ధన్యవాదములు
Köszönöm

Backup slides

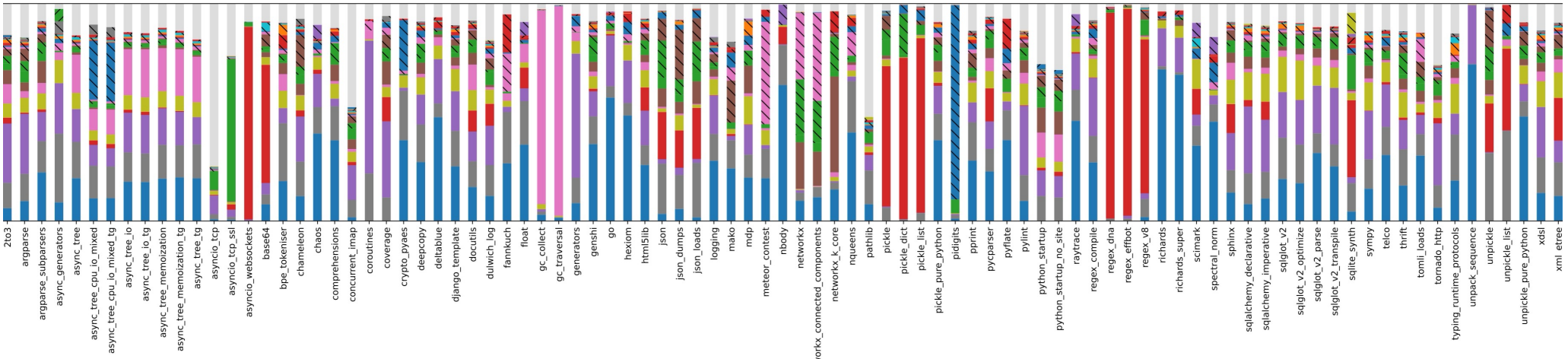
arm

The JIT graveyard

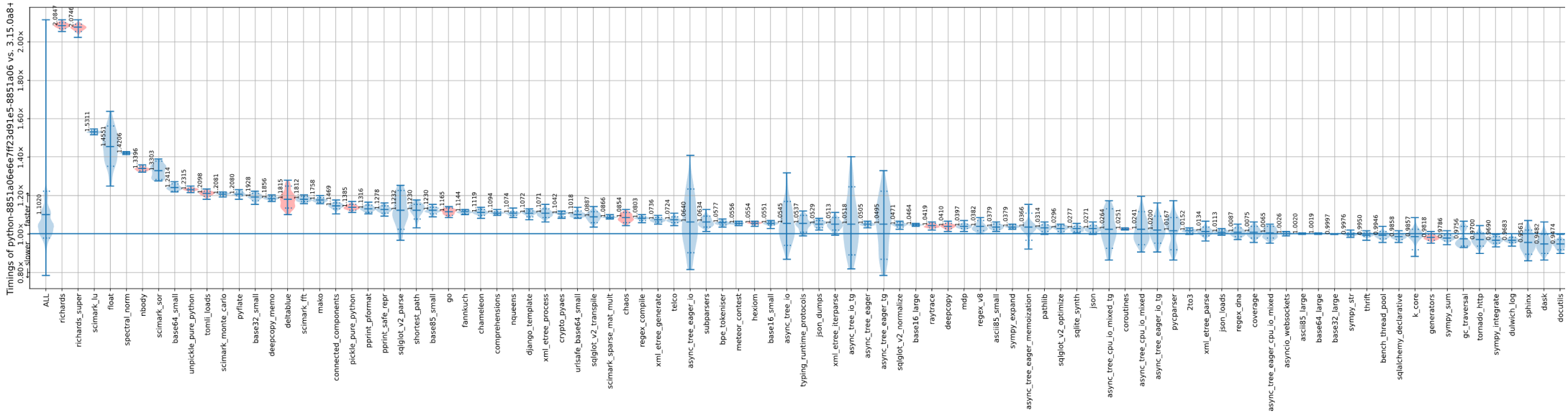
Many attempts. No universal solution.

- Psyco
  - An importable extension module that enabled some form of JIT of the eval loop calling CPython APIs. From the y2k Python 2.0 era. 32-bit x86 only. Python <= 2.6 only
- Unladen Swallow
  - Aimed to use LLVM to build a method-at-a-time JIT for CPython 2.6
- PyPy
- Pyston
  - Aimed to use LLVM to build a method-at-a-time JIT for CPython 2.6 (spot the similarity with Unladen Swallow) and re-implement the rest of the VM in C++.
- Skybison
  - Originally called Pyro, this was Instagram (now Meta)'s attempt at repeating history.
  - Re-implemented the VM in C++, including an interpreter written in assembly.
- S6
  - From an Alphabet research subsidary.

There is no silver bullet (details)



JIT impact across workloads (details)



try/finally: moving cost off the fast path (details)

Python 3.8

```
4           0 SETUP_FINALLY           10 (to 12)

5           2 LOAD_GLOBAL              1 (do_work)
            4 CALL_FUNCTION            0
            6 POP_TOP
            8 POP_BLOCK
           10 BEGIN_FINALLY

7     >>   12 LOAD_GLOBAL              0 (cleanup)
           14 CALL_FUNCTION            0
           16 POP_TOP
           18 END_FINALLY
           20 LOAD_CONST               0 (None)
           22 RETURN_VALUE
```

Python 3.9

```
4           0 SETUP_FINALLY           16 (to 18)

5           2 LOAD_GLOBAL              0 (do_work)
            4 CALL_FUNCTION            0
            6 POP_TOP
            8 POP_BLOCK

7          10 LOAD_GLOBAL              1 (cleanup)
           12 CALL_FUNCTION            0
           14 POP_TOP
           16 JUMP_FORWARD             8 (to 26)
     >>    18 LOAD_GLOBAL              1 (cleanup)
           20 CALL_FUNCTION            0
           22 POP_TOP
           24 RERAISE
     >>    26 LOAD_CONST               0 (None)
           28 RETURN_VALUE
```

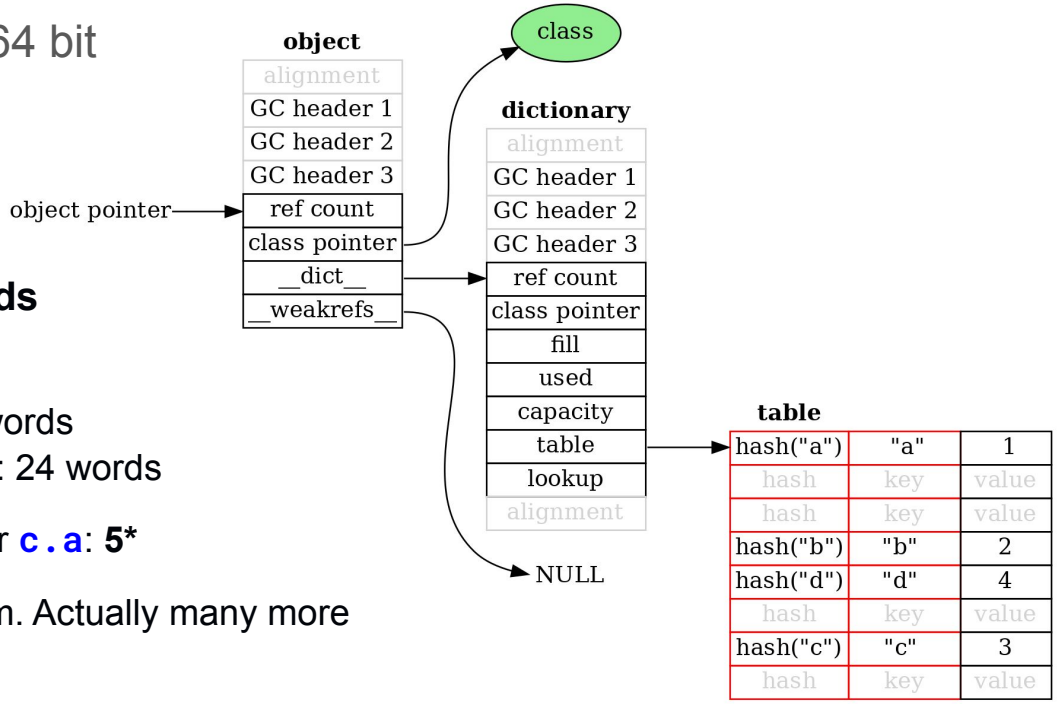
Python 3.15

```
  3           RESUME                   0

  4           NOP

  5   L1:     LOAD_GLOBAL              1 (do_work + NULL)
              CALL                     0
              POP_TOP

  7   L2:     LOAD_GLOBAL              3 (cleanup + NULL)
              CALL                     0
              POP_TOP
              LOAD_COMMON_CONSTANT     7 (None)
              RETURN_VALUE

  --  L3:     PUSH_EXC_INFO

  7           LOAD_GLOBAL              3 (cleanup + NULL)
              CALL                     0
              POP_TOP
              RERAISE                  0

  --  L4:     COPY                     3
              POP_EXCEPT
              RERAISE                  1
ExceptionTable:
  L1 to L2 -> L3 [0]
  L3 to L4 -> L4 [1] lasti
```


Object layout (pre 3.3)

Memory layout of Python objects (pre 3.3)

All sizes are for 64 bit



352 bytes. 44 words

- Object: 8 words
- Dictionary: 12 words
- Dictionary table: 24 words

Memory accesses for `c.a`: **5***

*Theoretical minimum. Actually many more

Object layout (3.3)

Memory layout of Python objects (3.3)

Share the keys and hashes

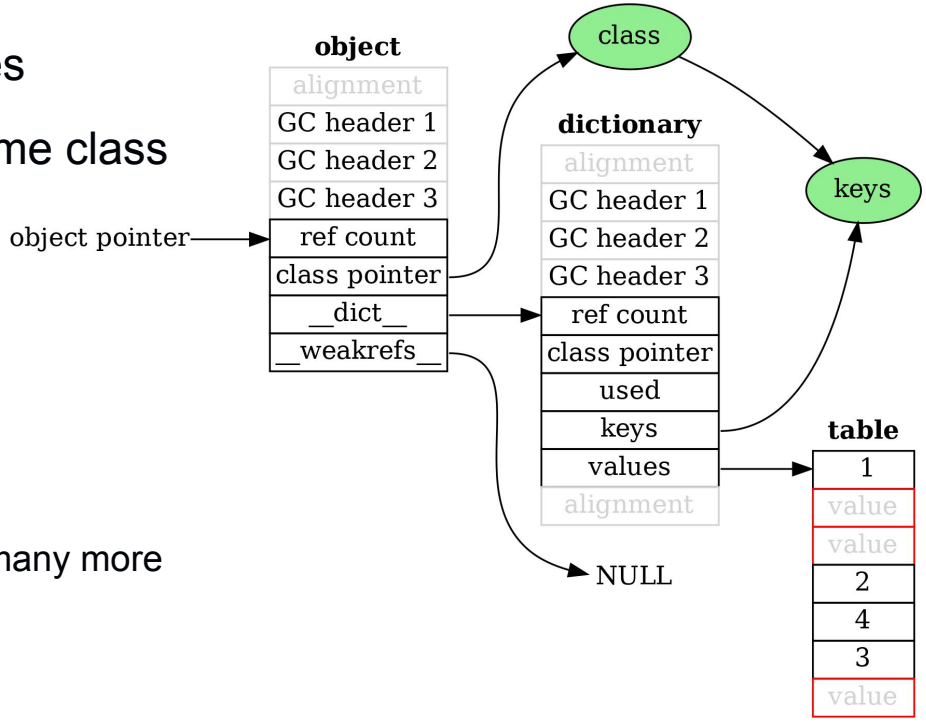
between objects of the same class

208 bytes. 26 words

- Object: 8 words
- Dictionary: 10 words
- Dictionary values: 8 words

Memory accesses for `c.a`: **6***

*Theoretical minimum. Actually many more



Object layout (3.6)

Memory layout of Python objects (3.6)

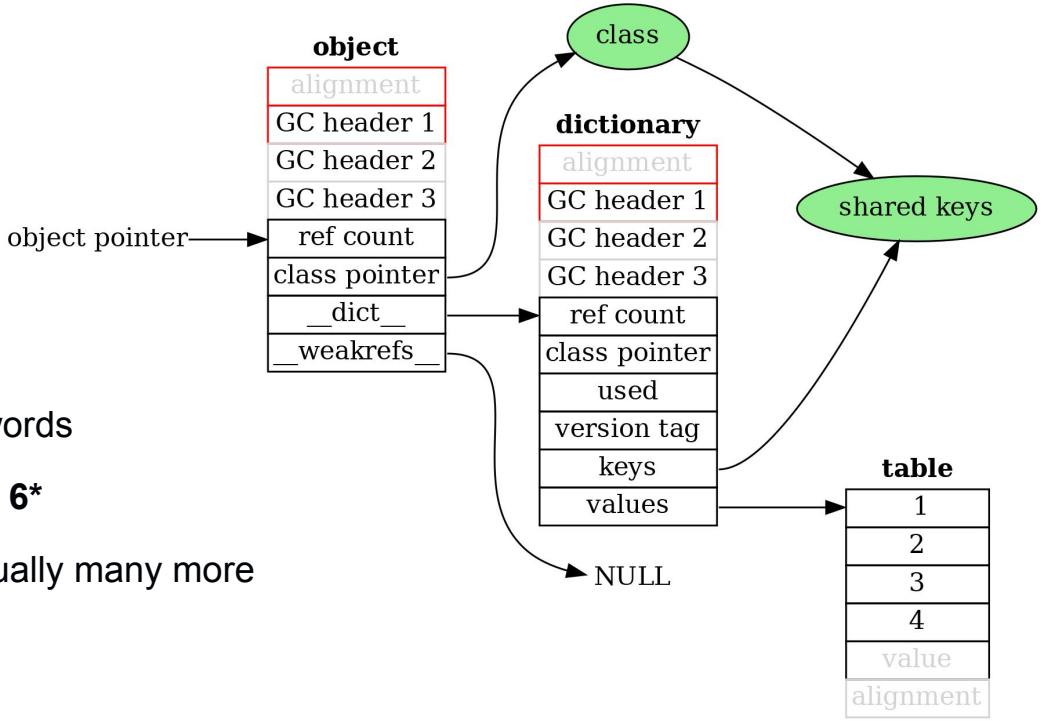
Compact dictionaries

192 bytes. 24 words

- Object: 8 words
- Dictionary: 10 words
- Dictionary values: 6 words

Memory accesses for `c.a`: **6***

*Theoretical minimum. Actually many more



Object layout (3.8)

Memory layout of Python objects (3.8)

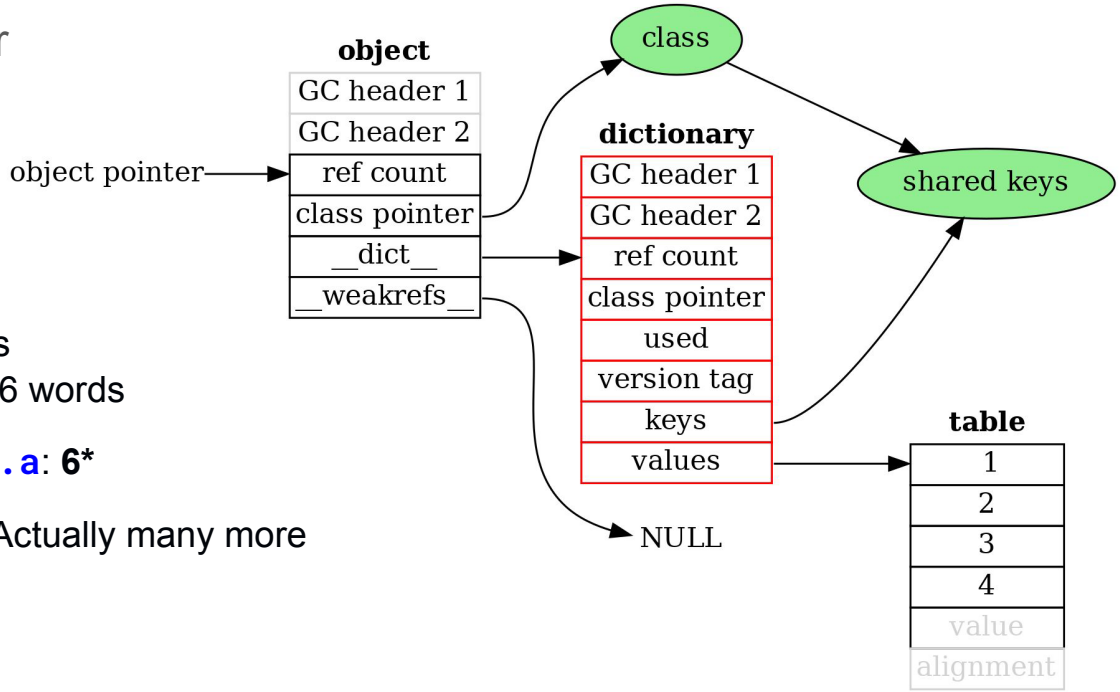
Smaller GC header

160 bytes. 20 words

- Object: 6 words
- Dictionary: 8 words
- Dictionary values: 6 words

Memory accesses for `c.a`: **6***

*Theoretical minimum. Actually many more



Object layout (3.11)

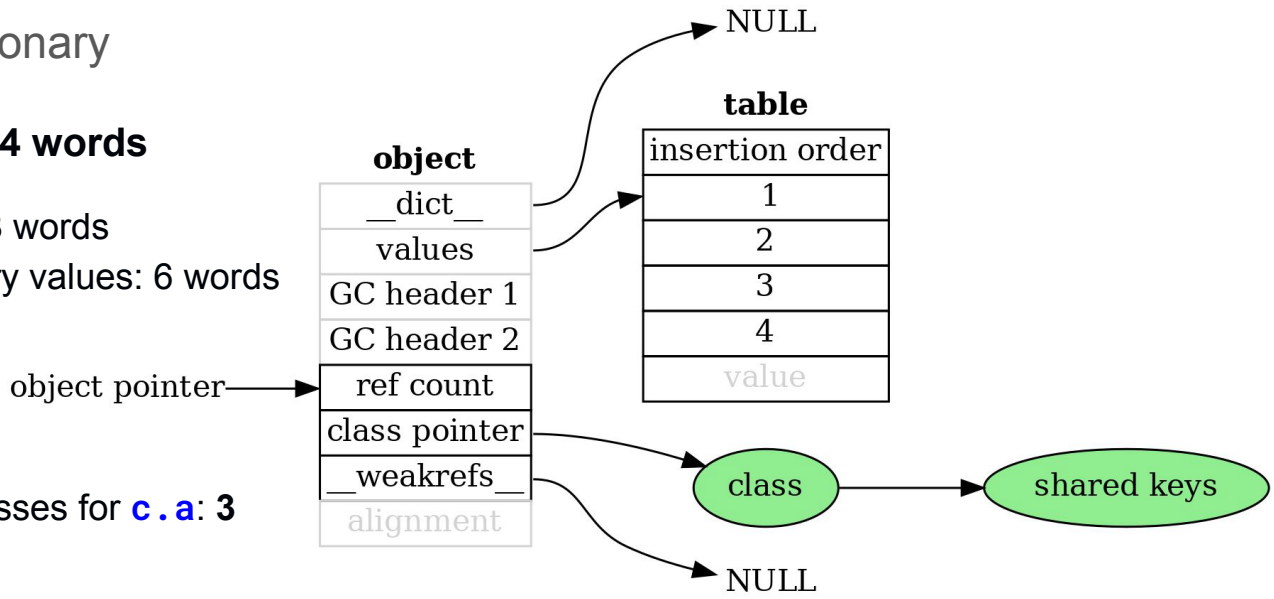
Memory layout of Python objects (3.11)

Virtual dictionary

112 bytes. 14 words

- Object: 8 words
- Dictionary values: 6 words

Memory accesses for `c.a`: **3**



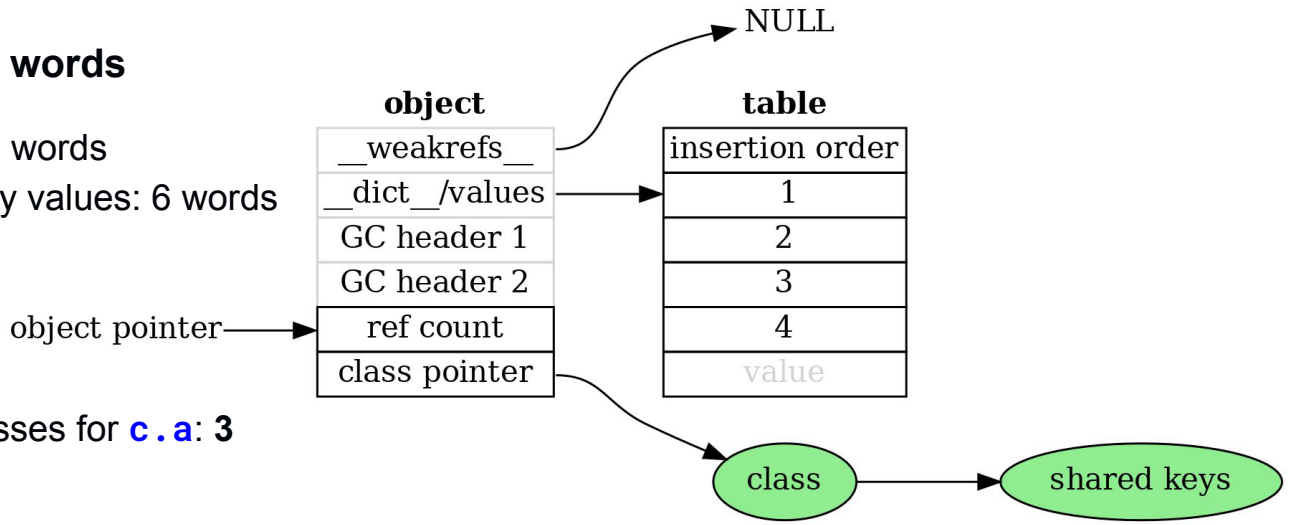
Object layout (3.12)

Memory layout of Python objects (3.12)

Move __weakrefs__ into pre-header

96 bytes. 12 words

- Object: 6 words
- Dictionary values: 6 words

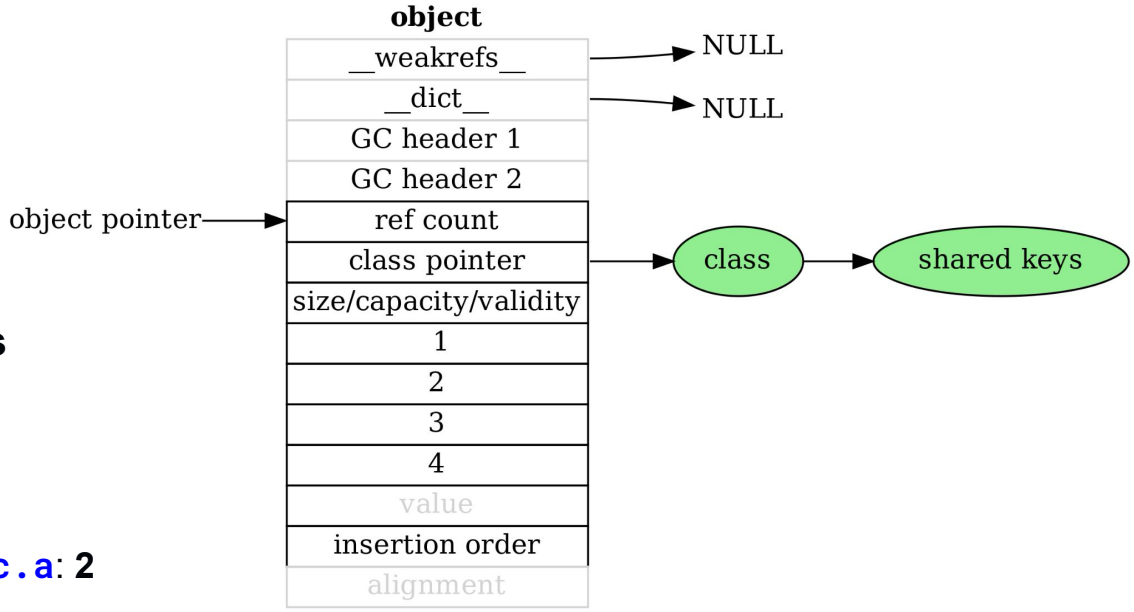


Memory accesses for `c.a`: **3**

Object layout (3.13)

Memory layout of Python objects (3.13)

Inline values



112 bytes. 14 words

- Object: 14 words

Memory accesses for `c.a`: **2**

PEP 703 — Key Techniques

- Atomic reference counting
- Deferred reference counting
- Biased reference counting
- Thread-local refcount buffers
- Immortal objects
- Lock-free / low-lock object access

Optimization Ladder